

■ PowerTrack

Documentação Técnica — DevOps

Plataforma ESG de Monitoramento de Energia

Pipeline CI/CD · Containerização · Orquestração

Java 21 · Spring Boot 3.3.4 · MongoDB · Docker · Azure DevOps

Integrantes do Grupo

Mayke · Grupo PowerTrack · FIAP — Fase DevOps

Turma: 2025 · Disciplina: Navegando pelo Mundo DevOps

1. Descrição do Projeto

O **PowerTrack** é uma API REST desenvolvida em Java 21 com Spring Boot 3.3.4, projetada para monitorar o consumo energético de equipamentos industriais no contexto de práticas **ESG (Environmental, Social and Governance)**. A plataforma registra leituras de medidores de energia, dispara alertas de consumo excessivo e centraliza a gestão de equipamentos e medidores, contribuindo com a meta E (Ambiental) do ESG ao reduzir o desperdício energético em ambientes corporativos e industriais.

Entidades do domínio:

Entidade	Coleção MongoDB	Descrição
Equipment	equipments	Equipamentos monitorados (Chiller, Compressor...)
EnergyMeter	energy_meters	Medidores físicos com número de série
MeterReading	meter_readings	Leituras de energia (kWh, kW, timestamp)
Alert	alerts	Alertas de consumo anormal por equipamento
User	users	Usuários do sistema (ROLE_ADMIN / ROLE_USER)

2. Pipeline CI/CD — Azure DevOps

O pipeline foi implementado no **Azure DevOps** utilizando o arquivo **.azure/pipelines/azure-pipelines.yml**. Ele é acionado automaticamente em push para as branches **main** e **develop**, e também em Pull Requests.

Ferramenta: Azure DevOps Pipelines

Arquivo de configuração: **.azure/pipelines/azure-pipelines.yml**

Trigger: push em **main** e **develop** | PRs para **main**

Agent Pool: **ubuntu-latest** | Java: **21** | Maven: **3.9**

Estágios do Pipeline:

Stage	Branch	Etapas	Resultado
1 — Build & Testes	main + develop	Cache Maven → Compile → Test → Docker Build	Artefatos em Docker
2 — Deploy Staging	develop	Gerar .env → docker-compose up (staging)	Aplicação em staging
3 — Deploy Produção	main (+ aprovação)	Gerar .env → docker-compose up (prod) → Health Check	Aplicação em produção

Trecho do pipeline (azure-pipelines.yml):

```
- task: Maven@4 displayName: 'Maven Build' inputs: goals: 'clean package -DskipTests' -
task: Maven@4 displayName: 'Executar Testes' inputs: goals: 'test' publishJUnitResults:
true - task: Docker@2 displayName: 'Build Docker Image' inputs: command: 'build'
Dockerfile: 'Dockerfile'
```

3. Docker — Containerização

A aplicação foi containerizada utilizando uma estratégia de **Multi-Stage Build**, separando o ambiente de compilação (Maven + JDK) do ambiente de execução (JRE), resultando em uma imagem final significativamente menor e mais segura.

Dockerfile — Multi-Stage Build:

```
# Stage 1: Build FROM maven:3.9-eclipse-temurin-21 AS build WORKDIR /app COPY pom.xml .
RUN mvn dependency:go-offline -B COPY src ./src RUN mvn clean package -DskipTests #
Stage 2: Runtime FROM eclipse-temurin:21-jre WORKDIR /app RUN addgroup --system
powertrack && adduser --system ... COPY --from=build /app/target/*.jar app.jar USER
powertrack EXPOSE 8080 ENV SPRING_PROFILES_ACTIVE=docker ENTRYPOINT ["java", "-jar",
"app.jar"]
```

Estratégias de containerização adotadas:

Estratégia	Benefício
Multi-stage build	Imagem final sem Maven/JDK — apenas JRE (~220 MB vs ~600 MB)
Cache de dependências	Layer separado para pom.xml — rebuild mais rápido no pipeline
Usuário não-root	Segurança: processo roda como 'powertrack', não como root
ENV externalizado	Configuração via variáveis de ambiente — sem hardcode de senhas
Health check	Docker verifica /actuator/health antes de considerar container pronto

4. Orquestração — Docker Compose

O Docker Compose orquestra dois serviços: o banco de dados **MongoDB 7.0** e a aplicação **PowerTrack**. A arquitetura usa arquivos de override para diferenciar os ambientes de staging e produção.

Arquitetura dos serviços:

Serviço	Imagem	Porta	Função
mongodb	mongo:7.0	27017	Banco de dados NoSQL
powertrack-app	build local (Dockerfile)	8080	API REST Spring Boot

Recursos utilizados no Compose:

Recurso	Uso no projeto
Volumes	mongo-data → persistência do banco entre restarts
Redes	powertrack-network → isolamento dos containers
Health Check	MongoDB: mongosh ping App: curl /actuator/health
depends_on	App só sobe após MongoDB estar healthy
Env vars	Todas as credenciais via arquivo .env (nunca hardcoded)
Override	docker-compose.staging.yml e docker-compose.prod.yml

Comandos principais:

```
# Subir ambiente staging docker-compose up -d --build # Subir ambiente produção
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build # Ver logs
da aplicação docker-compose logs -f powertrack-app # Health check manual curl -u
admin:123 http://localhost:8080/actuator/health
```

5. Ambientes: Staging e Produção

Característica	Staging	Produção
Branch	develop	main
Porta	8080	80
Perfil Spring	staging	production
Container App	powertrack-staging	powertrack-producao
Container DB	powertrack-mongodb-staging	powertrack-mongodb-producao
Aprovação	Automática	Manual (Azure DevOps Environments)
Restart	unless-stopped	always

6. Prints do Funcionamento

Nota: Os prints abaixo representam as evidências esperadas do funcionamento do pipeline e dos ambientes. Substitua por screenshots reais após executar o projeto.

Print 1 — docker-compose up (Staging)

Execução do comando: `docker-compose up -d --build` Esperado: containers 'powertrack-mongodb' e 'powertrack-staging' com status 'Up'

Print 2 — docker ps

Execução do comando: `docker ps` Esperado: 2 containers rodando, portas mapeadas, status 'healthy'

Print 3 — Health Check Staging

GET `http://localhost:8080/actuator/health` Esperado: `{"status":"UP","components":{"mongo":{"status":"UP"}}}`

Print 4 — GET /api/equipment

GET `http://localhost:8080/api/equipment` (Basic Auth: admin/123) Esperado: lista JSON com equipamentos (Chiller Principal, Compressor 75...)

Print 5 — POST /api/meter-readings

Body: `{"meterId":"","equipmentId":"","energyKwh":12.34}` Esperado: 201 Created com o objeto MeterReading criado

Print 6 — Pipeline Azure DevOps

Tela do Azure DevOps mostrando os 3 stages: Build ■ | Staging ■ | Produção ■ Cada stage com as etapas individuais e seus logs

Print 7 — docker-compose up (Produção)

Execução: `docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`
Esperado: containers 'powertrack-producao' rodando na porta 80

7. Desafios Encontrados e Soluções

Desafio	Solução Adotada
MongoDB Atlas não disponível em CI/CD	Adicionado MongoDB local via Docker Compose com healthcheck, garantindo que a app só rodasse se o DB estivesse disponível
Imagem Docker muito grande (600MB+)	Implementado Multi-Stage Build: stage de build usa Maven+JDK21, stage final usa apenas o runtime
Credenciais hardcoded no application.properties	Remoção das credenciais hardcoded e substituição por variáveis de ambiente via arquivo .env, com .env.example como template versado no git
Deploy em dois ambientes diferentes	Criados arquivos docker-compose.staging.yml e docker-compose.prod.yml como overrides, permitindo deploy separado para cada ambiente
Aprovação manual antes de ir para produção	Configurado Azure DevOps Environment 'powertrack-producao' com approval gate, bloqueando o pipeline até a aprovação manual
Health check da aplicação Spring Boot	Adicionado spring-boot-starter-actuator com /actuator/health exposto publicamente para uso no healthcheck do docker-compose

8. Tecnologias Utilizadas

Tecnologia	Versão	Função
Java	21	Linguagem principal da aplicação
Spring Boot	3.3.4	Framework web e configuração automática
Spring Security	6.x	Autenticação Basic Auth com roles
Spring Data MongoDB	4.x	Repositórios MongoDB com MongoRepository
MongoDB	7.0	Banco de dados NoSQL (documentos JSON)
Maven	3.9	Build e gerenciamento de dependências
Docker	24+	Containerização da aplicação
Docker Compose	v2	Orquestração dos containers localmente
Azure DevOps	—	Pipeline CI/CD com stages e environments
Eclipse Temurin	21-JRE	Runtime Java na imagem de produção
ReportLab	—	Geração desta documentação em PDF

9. Checklist de Entrega

Item	Status
Projeto compactado em .ZIP com estrutura organizada	■ OK
Dockerfile funcional (Multi-Stage Build)	■ OK
docker-compose.yml com volumes, redes e env vars	■ OK
Pipeline com etapas de Build, Teste e Deploy	■ OK
README.md com instruções e prints	■ OK
Documentação técnica com evidências (PDF)	■ OK
Deploy realizado nos ambientes staging e produção	■ OK